

WEEK 06 · TAILWIND CSS

THE PAGE WRITTEN IN CLASSES

Same CSS. Different place to put it. And the first week Claude writes code with you.

WHERE WE ARE

Week 6 of 12 (week 13 is the exam). You can build any layout — and last week it took a stylesheet of hundreds of lines.

THE NEXT 180 MINUTES

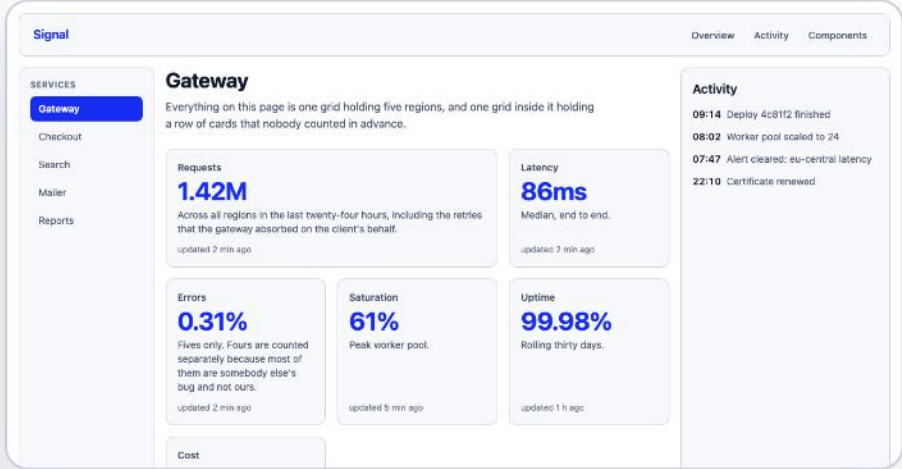
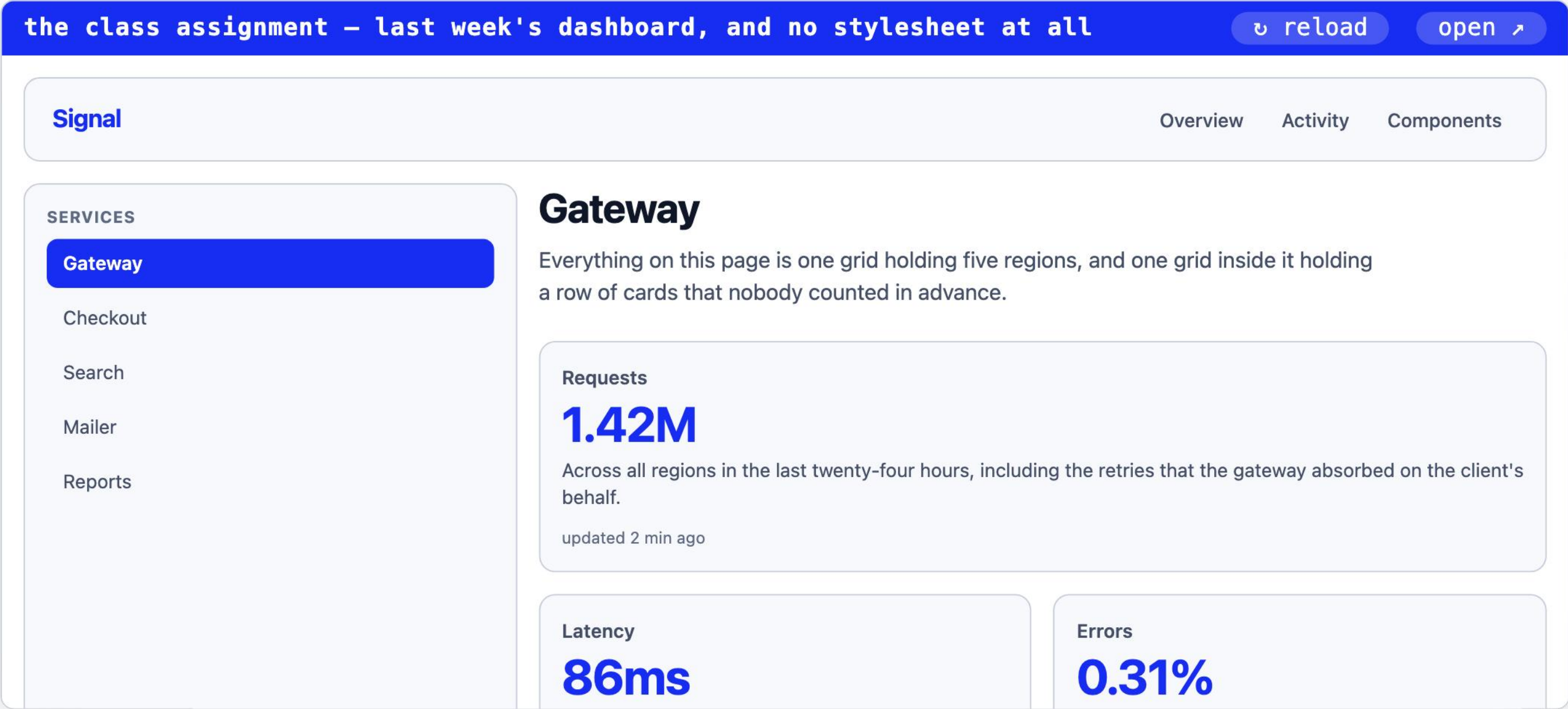
Five cycles, you building in every one: one card, translated · the shell in twelve columns · one link, four states, a second ramp · three answers to one prompt · one button, six copies. Break at 90, assignment at 145.

WHAT THIS UNLOCKS

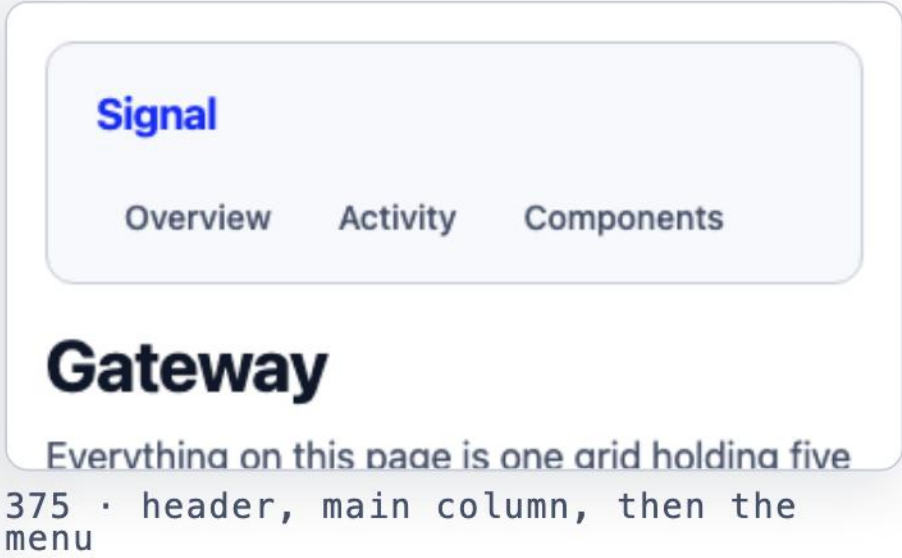
Every page from here, the project included — from a file beside the page, with the network off.

TARGET · MINUTE 0

THIS IS WHERE YOU WILL BE AT MINUTE 180



targets/week-06.png · 1280



375 · header, main column, then the menu

► **THE POINT** — the markup is last week's, element for element, and there is no stylesheet. Every class is a declaration you already know: a layout of three columns that becomes one on a phone, spacing from one scale, a focus ring you see only from the keyboard, a dark colour scheme that stays readable (4.5:1) in both modes, one definition per component. You build every piece of it today, once in a cycle and once in the assignment: you type nearly all of it, and the long blocks are marked **COPY THIS**.

WHY · THE PROBLEM FIRST

A STYLESHEET LIKE LAST WEEK'S HAS FOUR COSTS, AND YOU PAID THEM ALL

DEAD RULES

In the demo's notification card, over half the rules match nothing on the page. Nobody deleted them, and nobody was careless — you cannot prove a selector is unused by reading it.

INVENTED NAMES

Eleven in that one card.

`.notice__meta--compact` is not a design decision. It is bookkeeping you have to remember.

THE SPECIFICITY RATCHET

Three rules fighting over one colour. To change it you cannot write a rule — you have to write one that **wins**. And the next person has to beat yours.

And the fourth: two files, and your attention moving between them all day.

► Measured on a notification card's stylesheet: `demos/01-why-utilities/`, the optional slide after this one.

MEASURED: WHAT LAST WEEK'S STYLESHEET REALLY DID

a notification card, and its stylesheet, measured

reload

open

WRITTEN BY HAND — THE WAY WEEK 5 TAUGHT

! Certificate expires in 6 days

Renew it before Friday or the gateway starts refusing connections.

rules in the stylesheet	15
...matching nothing on this page	8
class names invented	11
rules that escalated	7
!important	1
worst specificity (a,b,c)	1,2,0

8 of 15 authored rules (53%) match nothing on this page.

The browser found them just now, by running `querySelectorAll` on every selector.

Dead:

`.notice--compact`

`.notice--compact .notice__icon`

`.notice--compact .notice__title`

THE SAME CARD, IN UTILITIES

rules in the stylesheet	0 – there is no stylesheet
...matching nothing on this page	0
class names invented	0
rules that escalated	0
!important	0
highest specificity possible	0,1,0

▶ **HANDS-ON** `demos/01-why-utilities/` — left column only. Count how many of those rules still do something. **DO NOT SCROLL RIGHT YET.**

WHY · THE IDEA IN ONE SLIDE

UTILITY CLASSES, AND WHAT UTILITY-FIRST MEANS

Utility class

A class that sets one thing:
`p-4` sets `padding: 1rem`.

Utility-first

You build the page out of them, in the HTML, with no stylesheet of your own.
Tailwind CSS provides them.

How they reach the page One script line, from a file beside your page (not from a CDN, a public server), reads the classes in your page and writes only the CSS they need.

```
<script src="vendor/tailwind.js"></script>
<p class="p-4 text-sm">...</p>
```

```
/* what the browser ends up with */
.p-4 { padding: 1rem }
.text-sm { font-size: .875rem }
```



AI THIS WEEK

ASSISTANT. IT WRITES; YOU CHOOSE, AND SAY WHY IN WRITING.

LAYER • WEEKS 6–7 • ASSISTANT

ALLOWED

Snippets, class lists, three versions of one component. Run every one where your project runs — `vendor/tailwind.js`, no network, 375 and 1280, Tab. Keep one, and write down what you rejected and why.

NOT ALLOWED

A line you cannot explain. A whole page. Anything off the course's stack — or off its setup (cycle 4 shows the trap). Keeping output without a written reason. **Cycle 1's you-do is AI-free, and so is cycle 3's.**

NO SUBSCRIPTION?

Everything works in the free chat, and nothing in the course depends on a paid plan. Cycle 4 is one prompt in three chats — whatever chat you have.

CYCLE 1 · BEFORE YOU TYPE

A UTILITY'S NAME IS THE PROPERTY, THEN THE VALUE

BOX

padding → p-* · px-* · py-* · pt-*
margin → m-* · mx-auto
border-radius → rounded-*
box-shadow → shadow-md

LAYOUT

display:flex → flex
repeat(3, 1fr) → grid-cols-3
gap → gap-* · gap-x-*

ALIGNMENT

justify-content → justify-*
align-items → items-*
align-self → self-*

TYPE

font-size → text-sm · text-lg
font-weight → font-semibold
text-decoration → underline ·
no-underline

COLOUR

color → text-*
background → bg-*
border-color → border-*

THE ONE TRAP

text-* means **both** size and colour. text-sm is a size; text-slate-600 is a colour. The value decides.

You do not memorise a list: you know the CSS, and the name follows the pattern. **x** is left and right, **y** is top and bottom, **t r b l** is one side.

BOX — SPACING, SIZE, BORDER

CSS YOU WROTE	UTILITY	NOTE
padding: 1rem	p-4	4 × the 0.25rem step
padding-inline: 1.5rem	px-6	py-* for block
padding-top: .5rem	pt-2	pt, pr, pb, pl
margin: 0	m-0	-mt-4 is negative
margin-inline: auto	mx-auto	the centring one
margin-block-start: auto	mt-auto	pushes to the bottom of a flex column
width: 100%	w-full	w-screen, w-fit, w-4
max-width: 36rem	max-w-xl	named steps, not the spacing scale
border: 1px solid	border	border-2, border-t
border-radius: .75rem	rounded-xl	rounded-full for a pill
box-sizing: border-box	—	already the default. It is in the reset.

LAYOUT — FLEX AND GRID

CSS YOU WROTE	UTILITY	NOTE
<code>display: flex</code>	<code>flex</code>	<code>inline-flex</code> , <code>grid</code> , <code>block</code> , <code>hidden</code>
<code>flex-direction: column</code>	<code>flex-col</code>	<code>flex-row</code> is the default
<code>flex-wrap: wrap</code>	<code>flex-wrap</code>	grid never needs it
<code>flex: 1 (= 1 1 0%)</code>	<code>flex-1</code>	equal shares, whatever the text. <code>flex-auto</code> is <code>1 1 auto</code>
<code>flex-basis: 0</code>	<code>basis-0</code>	after <code>flex-1</code> it adds nothing — measured
<code>gap: 1rem</code>	<code>gap-4</code>	<code>gap-x-*</code> , <code>gap-y-*</code>
<code>grid-template-columns: repeat(3, 1fr)</code>	<code>grid-cols-3</code>	up to <code>grid-cols-12</code>
<code>grid-column: span 2</code>	<code>col-span-2</code>	<code>col-start-*</code> , <code>col-end-*</code>
<code>grid-template-areas</code>	no utility	use the 12-column form, or brackets
<code>min-width: 0</code>	<code>min-w-0</code>	week 4's overflow fix, unchanged
<code>position: sticky</code>	<code>sticky</code>	with <code>top-0</code>

TYPE AND COLOUR

CSS	UTILITY
font-size: .875rem	text-sm
font-size: 1.5rem	text-2xl
font-weight: 600	font-semibold
line-height: 1.25	leading-tight
text-align: center	text-center
text-decoration: none	no-underline
text-transform: uppercase	uppercase
font-variant-numeric	tabular-nums

CSS	UTILITY
color: ...	text-brand-500
background: ...	bg-surface
border-color: ...	border-edge
outline-color: ...	outline-brand-500
opacity on a colour	bg-white/15
opacity: .5	opacity-50

text-* is both. **text-sm** is a size, **text-slate-600** is a colour. The value decides.

NOT SURE OF A NAME? ASK THE BROWSER

CSS in, utility out – by measurement, not by table

↺ reload

open ↗

UTILITY → CSS

a class, exactly as you would write it in the markup

p-4

p-4

md:px-8

hover:bg-brand-700

dark:bg-slate-900

flex

gap-4

rounded-xl

text-huge

```
.p-4 {  
  padding: calc(var(--spacing) * 4);  
}
```

...and that is all it is. One class, one rule.

applied to a real element, live:

specimen

CSS → UTILITY

a declaration, straight out of last week's stylesheet

padding: 1rem;

padding: 1rem;

display: flex;

gap: 1rem;

justify-content: space-between;

font-weight: 600;

padding: 13px;

max-width: 62ch;

you asked for:

padding-top: 16px

padding-right: 16px

padding-bottom: 16px

padding-left: 16px

these classes resolve to exactly that:

p-4

Checked by MEASURING, not by looking up a table: each one was applied to a real element and its computed value compared with yours. That is why the answer is trustworthy even for a property nobody anticipated.

► **HANDS-ON** `demos/03-translator/` — start with the **RIGHT** box. `padding: 1rem`, then `display: flex`, then `justify-content: space-between`. Stop before the fourth.

CYCLE 1 · BEFORE YOU TYPE

THE NUMBERS IN THE NAMES ARE STEPS ON A SCALE

SPACING SCALE — STEP ×

`0.25rem`

`0 · 0.5 · 1 · 1.5 · 2 · 3 ·`
`4 · 5 · 6 · 8 · 10 · 12 · 16`
`· 20 · 24`

Seeing a number in old CSS? **Divide by four.** `16px` is `4`.

TYPE SCALE — NAMED, NOT NUMBERED

`xs · sm · base · lg · xl ·`
`2xl · 3xl · 4xl ...`

Each carries a line-height with it.
Override with `leading-*`.

COLOUR RAMP — 50 → 950

`brand-100 · brand-300 ·`
`brand-500 · brand-700`

Higher is darker (`brand` is our colour, defined on the next slide).
Which steps are legal for text is a contrast question, not a palette one.

Stay on the steps. A few of them, used again and again, is what makes a page look designed.

THE SCALES ARE THE PRODUCT.

THE SHORT NAMES ARE A SIDE EFFECT.

spacing, type and colour, drawn at their real sizes

↺ reloadopen ↗

SPACING — EVERY STEP IS --SPACING TIMES A WHOLE NUMBER

w-0.5	2px	
w-1	4px	
w-1.5	6px	
w-2	8px	
w-3	12px	
{w-[13px]}	13px	
w-4	16px	
w-5	20px	
w-6	24px	
w-8	32px	
w-10	40px	
w-12	48px	
w-16	64px	
w-20	80px	
w-24	96px	

TYPE

text-xs	12px	Renew the certificate
text-sm	14px	Renew the certificate
text-base	16px	Renew the certificate

▶ **HANDS-ON** demos/04-scale/ — cover the readout and decide: **WHICH ONE DOES NOT BELONG?**

ONE NUMBER MOVES THE WHOLE PAGE

```
1 @theme {  
2   --spacing: 0.25rem;  
3 }
```

`p-4` is not “sixteen pixels”. It is **four units**, and the unit is one line.

What it is

A single multiplier behind every padding, margin, gap, width and height.

Key trait

Change it and everything moves together, staying in proportion.

Use it for

Denser layouts, a bigger touch target, a design that felt cramped.

Tip

This is week 3's token argument with a system on top. Same idea, more reach.

CYCLE 1 · BEFORE YOU TYPE

@theme IS tokens.css WITH A SECOND JOB

WEEK 3 — :root, YOUR TOKENS

```
:root {  
  --color-accent: hsl(234 89% 52%);  
}  
  
.brand { color: var(--color-accent) }
```

WEEK 6 — THE SAME VALUES, IN <STYLE TYPE="TEXT/TAILOWINDCSS">

```
@theme {  
  --color-brand-500: hsl(234 89% 52%);  
  --color-surface: hsl(222 44% 98%);  
  --color-edge: hsl(222 20% 88%);  
  --color-ink-muted: hsl(222 14% 40%);  
}
```

A **@theme** entry is a custom property, and also generates a family of classes. The prefix picks the family: **--color-** makes **text-**, **bg-** and **border-**.

CYCLE 1 · WHERE IT ENDS

THE CARD YOU ARE ABOUT TO BUILD, WITH NO STYLESHEET AT ALL

```
<article class="flex max-w-sm flex-col gap-2 rounded-xl
            border border-edge bg-surface p-4">
  <h2 class="m-0 text-lg font-semibold text-brand-500">Paris, Texas</h2>
  <p class="m-0 text-sm text-ink-muted">Thursday 20:00 · Hall B</p>
  <a class="self-start rounded-lg bg-brand-500 px-4 py-2
            font-semibold text-white no-underline" href="#book">Book a seat</a>
</article>
```

DEAD RULES: STRUCTURALLY ZERO

Not "fewer". There is no stylesheet, so there is nothing that **can** die.

NAMES INVENTED FOR STYLING: NONE

Nothing to remember, nothing to defend in review, nothing to rename.

SPECIFICITY: 0,1,0

Every utility is one class. A variant such as `hover:` adds a pseudo-class, 0,2,0. Nothing to escalate.

CYCLE 1 · BEFORE YOU TYPE

SQUARE BRACKETS, AND THE ONE QUESTION TO ASK FIRST

Square brackets write **any** value — an **arbitrary value**: `w-[728px]`.

Would you want a second element to have this exact value?

NO → BRACKETS

A 728×90 ad slot. A `mask-image` used once in a career. Nobody chose it and nobody will reuse it.

YES → `@theme`

It is a decision. Give it a name, and it becomes a first-class utility.

ALREADY ON THE SCALE → THE SMELL

`p-[13px]` when `p-3` exists. You are overriding a system that was working, one pixel at a time.

Two honest brackets this week: `max-w-[62ch]` (a measure in `ch`) and `grid-cols-[repeat(auto-fit,minmax(11rem,1fr))]` (week 5's track list, no spaces).

ARBITRARY VALUES – SYNTAX, AND THE TEST

FORM	EXAMPLE	WHEN
arbitrary value	<code>w-[728px]</code>	a size somebody else decided
arbitrary property	<code>[mask-image:linear-gradient(...)]</code>	a property with no utility at all
underscores for spaces	<code>grid-cols-[repeat(auto-fit, minmax(11rem,1fr))]</code>	a pattern a utility cannot take
a theme value	<code>bg-(--color-brand-500)</code>	rarely — the named class is better

“WOULD A SECOND ELEMENT WANT THIS EXACT VALUE?”

No → brackets are correct. Nobody chose it and nobody will reuse it.

YES

It is a decision. Put it in `@theme` and it gets a name.

ALREADY ON THE SCALE, OR ONE STEP OFF

This is the smell. `p-3` exists; `p-[13px]` is you losing.

MISCONCEPTION 4

DEEPER · SELF-STUDY

SQUARE BRACKETS: A TEST, NOT A TABOO

three legitimate uses and one smell, audited

↺ reload

open ↗

The test, and it is mechanical:

Would you want a *second* element to have this exact value?

yes → it is a decision. Give it a name in @theme.

no → it is a one-off. Brackets are the right answer.

And then: is it already on the scale, or within one step of it? If so you are overriding a system that was working, and that is the smell.

CORRECT A SIZE SOMEBODY ELSE DECIDED

A leaderboard slot is 728×90 because the ad network says so. Nobody on this team chose it, nobody will reuse it, and rounding it to the scale would break the thing. Brackets, no apology.



```
class="w-[728px] h-[90px] ..."
```

measured 0 x 0 – exactly the number the ad network specified.

Nobody would want a SECOND element at 728x90, so there is nothing to name.

The test says brackets, and the test is right.

▶ **HANDS-ON** demos/08-arbitrary/ — third panel first. Ask yourself: **HOW MANY COLOURS ARE IN THIS ROW?**

CYCLE 1 · 27 MIN

ONE CARD, TRANSLATED

A card written in ordinary CSS becomes classes, declaration by declaration, until its `<style>` block is empty and deleted — and the card has not moved. On the way: the reset Tailwind brings with it, `@theme` as week 3's tokens, and the one attribute that decides whether a token exists at all.

1 I DO

13 MIN

6 on the slides above, 7 in the editor · live-code/01-card-translated · the script line · `@theme` in the wrong block, then in its own · the card, the text, the button · the plain `<style>` deleted

2 WE DO

5 MIN

you type it on your copy, the tokens straight into `type="text/tailwindcss"` · compare with checkpoint 1

3 YOU DO

7 MIN

warmup/ — match the box until the judge says 0 · demos/03-translator, and divide by four

4 DEBRIEF

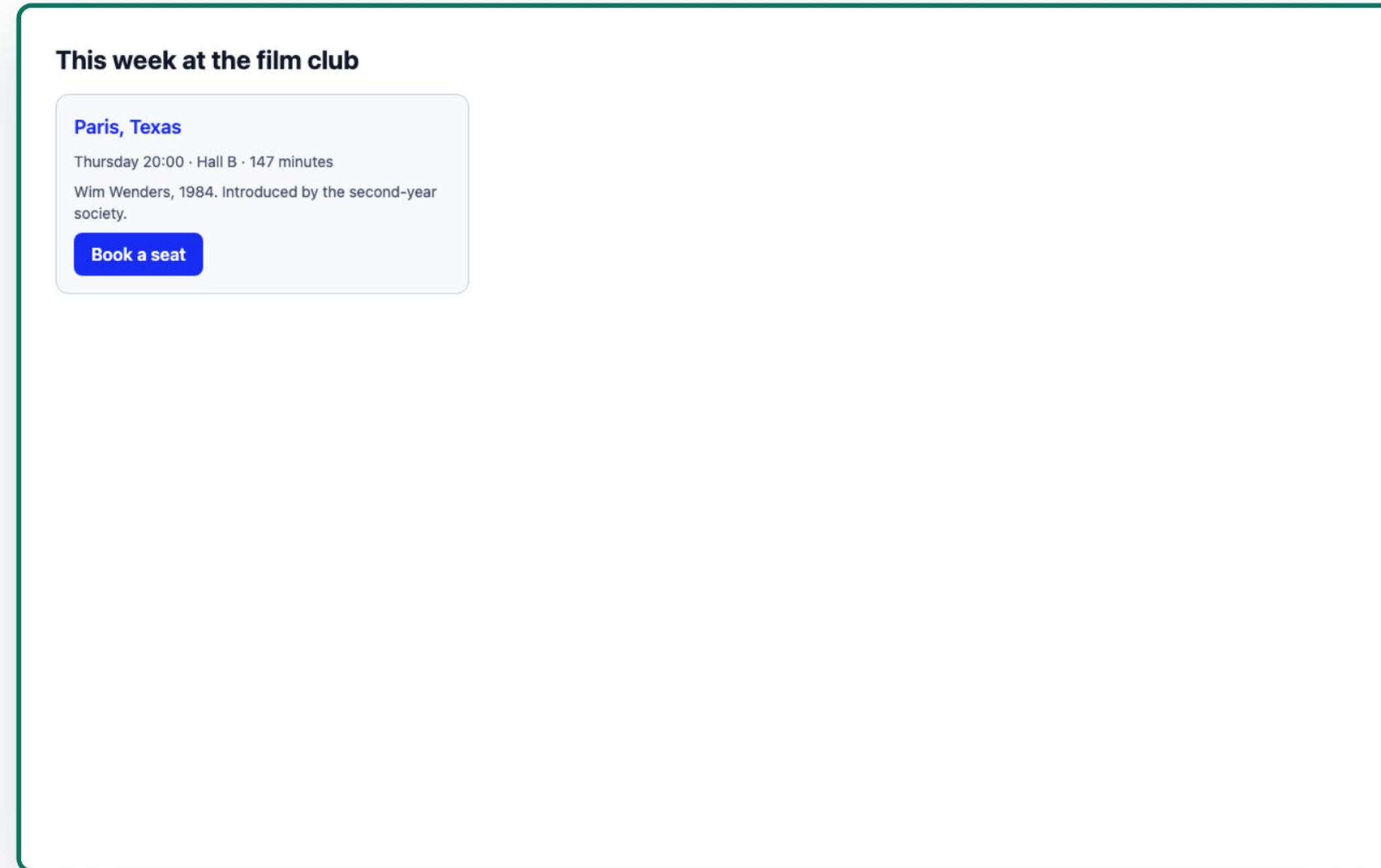
2 MIN

a class with no rule is silent · `p-[16px]` against `p-4` · deeper slides

YOU DO · AI-FREE

CHECKPOINT 1

YOUR SCREEN SHOULD LOOK LIKE THIS



Compare — `live-code/01-card-translated/checkpoint.png` is this picture at 1280; `live-code/01-card-translated/end/` is the code behind it. The card 384 wide, the title in the brand colour, and one `<style>` left in the page — the one with `type="text/tailwindcss"`.

AND IT NEVER FIGHTS THE CSS YOU DO WRITE

```
1 @layer theme, base, components, utilities; /* Tailwind, first thing */
2
3 @layer base { h2 { font-size: 1.5rem } } /* all of it: LAYERED */
4
5 h2 { font-size: 13px } /* yours is not – and it wins */
```

A layered rule loses to an unlayered one whatever the order. Your CSS beats the reset with no **!important**.

PREFLIGHT: THE RESET YOU ALREADY WROTE

WHAT THE SCRIPT LINE ALONE DOES	MEASURED	YOU MET IT IN
<code>box-sizing: border-box</code> on everything	the cycle-1 card: 418 to 384 wide	week 3's three lines
headings lose size and weight	<code>h1</code> : 32px bold, now 16px and 400	you give them back in classes
margins on headings and paragraphs: 0	<code>h1, p</code> : margin 0	—
a system font stack, line-height 1.5	Times becomes the system sans	week 3's <code>system-ui</code>
lists without bullets or indent	<code>ul</code> : list-style none, padding 0	week 4's nav list
images block, <code>max-width: 100%</code>	<code>img</code> : display block	week 5's images
buttons and links inherit	button background transparent, link colour inherited	week 3's <code>font: inherit</code>

All of it sits in `@layer base`. Your own unlayered CSS beats it whatever the order — which is why the cycle-1 card did not move until you deleted its rules.

CYCLE 2 · BEFORE YOU TYPE

A PREFIX IS A WIDTH CONDITION: **md :** MEANS 768PX AND WIDER

PREFIX	GENERATES	ROUGHLY
(none)	no query at all	every width. Start here.
sm:	@media (width >= 40rem)	640px
md:	@media (width >= 48rem)	768px
lg:	@media (width >= 64rem)	1024px
xl:	@media (width >= 80rem)	1280px
2xl:	@media (width >= 96rem)	1536px
max-md:	@media (width < 48rem)	the escape hatch. Not a default.

MOBILE-FIRST: THEY STACK, THEY DO NOT TAKE TURNS

No prefix is the phone. At 1400px, `sm`, `md`, `lg` and `xl` all match, and the last one wins. None of them fights.

A BREAKPOINT IS THE WIDTH WHERE THE LAYOUT CHANGES

They are written in `rem`, so a reader who enlarged the browser font gets them earlier — which is correct.

WEEK 5'S RULE, NOW IMPOSSIBLE TO GET BACKWARDS

which breakpoints match, at any width you like

reloadopen

frame width

1400px

smmdlgxl2xl

Requests

1.42M

Latency

86ms

Errors

0.31%

WHICH BREAKPOINTS MATCH, RIGHT NOW

sm:	≥ 40rem (640px)	matches – not used in this markup
md:	≥ 48rem (768px)	matches
lg:	≥ 64rem (1024px)	matches – not used in this markup
	≥ 80rem	

▶ **HANDS-ON** demos/05-responsive/ — start at 1400 and drag **DOWN**, slowly. Stop at exactly 768 and predict what happens at 767.

THERE IS NO `max-*` BY DEFAULT, AND THAT IS A DECISION

A MAX-WIDTH QUERY

States what a thing **stops** being.
Every one has to be undone by the next size up — which is the tangle last week's assignment made you untangle.

A PREFIX

Only ever **adds**. Four of them can match at once and none of them fights. Mobile-first is not a discipline here; it is the shape of the tool.

THE ESCAPE HATCH EXISTS

`max-md:` is real, because a system with no escape hatch is one you abandon. It is still the wrong default. **Remember that it exists.**

CYCLE 2 · BEFORE YOU TYPE

YOU WROTE THIS LAST WEEK. IT IS THE SAME RULE.

WEEK 5 — BY HAND

```
.kpi {
  display: grid;
  grid-template-columns: 1fr;
}

@media (min-width: 48em) {
  .kpi {
    grid-template-columns: repeat(3, 1fr);
  }
}
```

WEEK 6 — AND IT GENERATES THE SAME QUERY

```
<section class="grid grid-cols-1
              md:grid-cols-3">
```

Nine lines and an invented class name become one attribute. The **breakpoint did not move**: `48em` and Tailwind's `48rem` are both 768px at the default font size — you can read the query back out of the browser and check.

CYCLE 2 · BEFORE YOU TYPE

AND THE 12-COLUMN SHELL IS ARITHMETIC YOU ALREADY KNOW

WEEK 5, BY HAND – THE CSS UNDERNEATH

```
.shell {  
  display: grid;  
  grid-template-columns: repeat(12, 1fr);  
}  
  
nav { grid-column: span 4 }  
main { grid-column: span 8 }
```

WEEK 6, AND THE ASSIGNMENT'S SHELL

```
<div class="grid md:grid-cols-12">  
  <nav class="md:col-span-4 xl:col-span-2">  
  <main class="md:col-span-8 xl:col-span-7">  
  <aside class="md:col-span-12 xl:col-span-3">  
</div>
```

col-span-4 takes four of the twelve columns. Twelve, because twelve divides by 2, 3, 4 and 6 – every design tool speaks it.

CYCLE 2 · BEFORE YOU TYPE

order CHANGES WHERE AN ITEM IS PAINTED

WEEK 4 – THE CSS

```
.a { order: 1 }    /* painted later */  
.b { order: -1 }   /* painted earlier */  
/* equal order: source order decides */
```

WEEK 6 – THREE CLASSES YOU WILL MEET

```
order-first      /* order: -9999 */  
order-last       /* order: 9999 */  
md:order-none    /* order: 0, from 768 up */
```

Items are painted lowest **order** first, and **order-first** beats every item in the grid.

CYCLE 2 · BEFORE YOU TYPE

flex-1 IS ALREADY A ZERO BASIS

ON EACH OF THREE TILES COMPUTED FLEX

WIDTHS AT 1280 (CYCLE 2)

nothing (inside flex) 0 1 auto

69 · 473 · 73 — each as wide as its text wants

flex-auto 1 1 auto

69 · 473 · 73 — the free space shared, the text first

flex-1 1 1 0%

205 · 205 · 205

flex-1 basis-0 1 1 0px

205 · 205 · 205 — no change

WHY

Tailwind 4 compiles flex-1 to flex: 1, and the CSS shorthand with one number sets the basis to 0%. The grow factor then shares **all** of the row.

WHAT TO WRITE ABOUT IT

A class that changes nothing is still a class somebody has to read. In a review, "basis-0 is redundant after flex-1" is a sentence about the code — with a measurement behind it.

CYCLE 2 · 25 MIN

THE SHELL, IN TWELVE COLUMNS

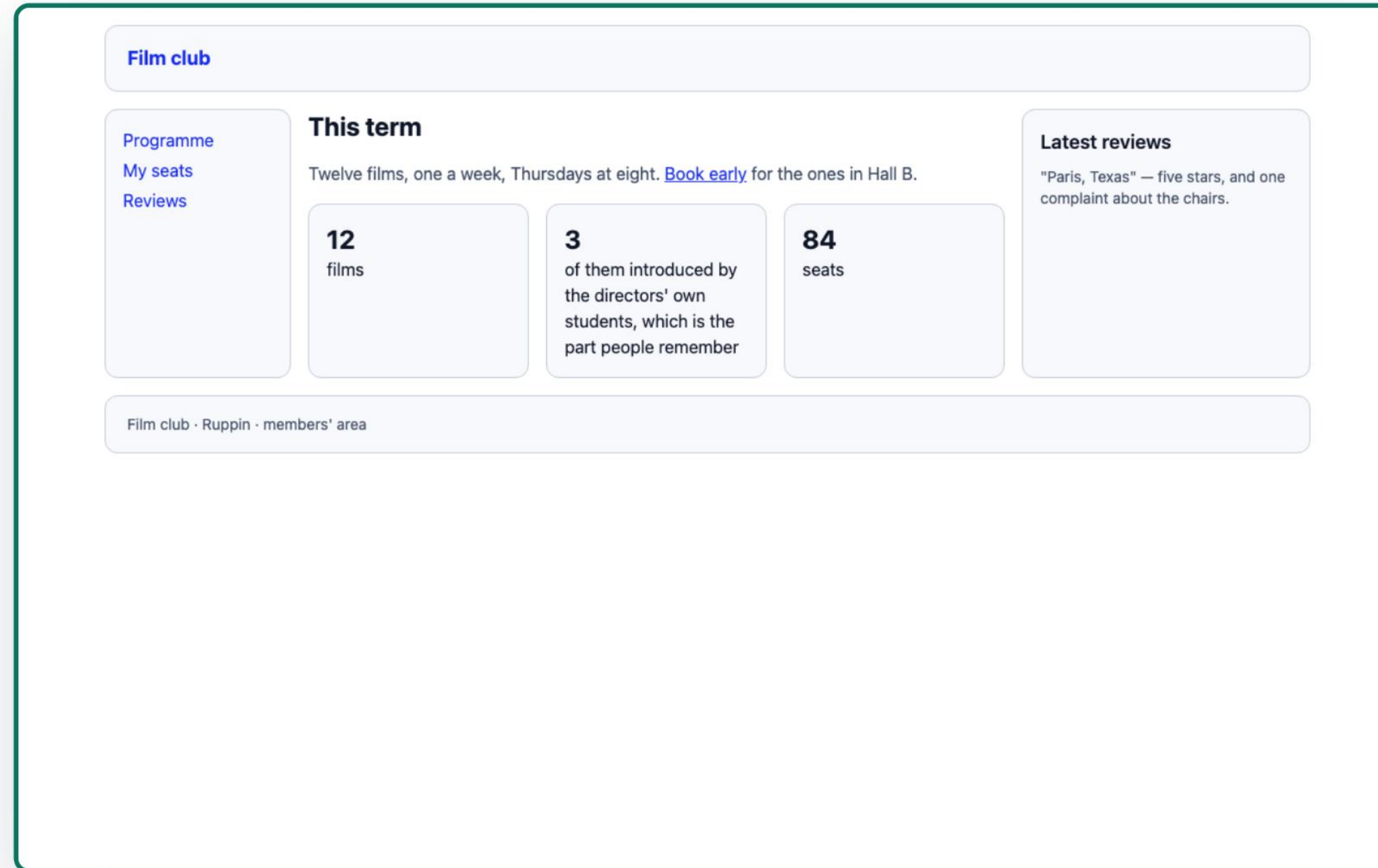
Five regions, three arrangements, and not one media query in the file: one column on a phone, two at 768, three at 1280. Then the one utility that puts the main column first on a phone — and puts it before the header too, until you read what "first" means.

1 I DO12 MIN 5 on the slides above, 7 in the editor · live-code/02-shell-twelve-columns · grid-cols-1 · md:grid-cols-12 and the spans · xl: · order-first, and what it does to the header · md:order-none · Tab at 375 · flex-1 on the stats	2 WE DO5 MIN you type it on your copy, order-first on both regions from the start · compare with checkpoint 2 at 1280, then drag to 375	3 YOU DO6 MIN one more width, lg:, and a region that moves · AI as assistant: a class list is allowed, a class you cannot explain is not · the one self-check line is copy this	4 DEBRIEF2 MIN first of every item · Tab follows the DOM · flex-1 already means basis 0 · order and the Tab key · deeper slides
--	---	---	---

YOU DO · AI AS ASSISTANT

CHECKPOINT 2

YOUR SCREEN SHOULD LOOK LIKE THIS



Compare — `live-code/02-shell-twelve-columns/checkpoint.png` is this picture at 1280; `live-code/02-shell-twelve-columns/end/` is the code behind it. Three columns at 1280 and three equal tiles (205 each); at 375 the header, then the content, then the menu. No `@media` and no `max-` in the file.

CYCLE 2 · AFTER YOU TYPED IT

order MOVES THE PAINT, NOT THE TAB KEY

CYCLE 2, AT 375 – PAINTED TOP TO BOTTOM, AND THE TAB ORDER

```
<header class="order-first md:order-none"> y 16
<main   class="order-first md:order-none"> y 94
<nav>                                     y 540
Tab: menu links (y 560) first, Book early (y 169) last
```

What it is

`order-first` is

`order: -9999`: first of **every** item. Equal orders keep source order.

What it is not

A change to the document. Tab and a screen reader follow the DOM.

When it is fine

Neighbouring navigation and content. Never where the order **is** the meaning — a form, a sequence.



CYCLE 3 · BEFORE YOU TYPE

STATES, AND NO JAVASCRIPT IN ANY OF THEM

hover · focus-visible · active, and two more that look like JavaScript↶ reloadopen ↗

hover: idle

point at me

@media (hover: hover) {
 .hover\:bg-brand-700:hover {
 background-color: var(--color-brand-700);
 }
}

Notice the media query. This is why hover never sticks on a phone.

peer-checked: idle

☐ email me when a deploy fails

.peer-checked\:text-brand-500:is(:where(.peer):checked ~ *) {
 color: var(--color-brand-500);
}

focus-visible: idle

reach me with Tab

.focus-visible\:outline-brand-500:focus-visible {
 outline-color: var(--color-brand-500);
}

The one you cannot demonstrate with a mouse, and the one always graded.

active: idle

hold the button down

.active\:bg-brand-700:active {
 background-color: var(--color-brand-700);
}

Lasts exactly as long as the mouse button is down. Rarely enough on its own.

group-hover: idle

Gateway
hover anywhere on the card — this line changes

@media (hover: hover) {
 .group-hover\:text-brand-500:is(:where(.group):hover *) {
 color: var(--color-brand-500);
 }
}

Parent hovered, child styled. Zero JavaScript — read the selector.

▶ **HANDS-ON** `demos/06-variants/` — do the first three with the mouse, then **PUT THE MOUSE DOWN** and reach the fourth with Tab.

READ THE SELECTOR AND THE MAGIC EVAPORATES

```
1 /* group-hover:text-brand-500 – inside hover: hover, like every hover variant */
2 @media (hover: hover) {
3   .group-hover\:text-brand-500:is(:where(.group):hover *) { ... }
4 }
5
6 /* peer-checked:text-brand-500 */
7 .peer-checked\:text-brand-500:is(:where(.peer):checked ~ *) { ... }
```

“INSIDE A HOVERED `.group`”

A descendant combinator and a pseudo-class. Week 3 taught both.

“A LATER SIBLING OF A CHECKED `.peer`”

The `~` combinator — which is why the checkbox must come **first** in the markup. CSS cannot look backwards.

THE TWO THAT LOOK LIKE THEY NEED JAVASCRIPT

VARIANT	GENERATES	USE IT FOR
<code>group-hover:</code>	<code>:is(:where(.group):hover *)</code>	parent hovered, child styled. Put <code>group</code> on the parent
<code>peer-checked:</code>	<code>:is(:where(.peer):checked ~ *)</code>	a control styling a later sibling. Put <code>peer</code> on the control

READ THEM OUT LOUD

“an element inside a hovered `.group`”, and “a later sibling of a checked `.peer`”. A combinator and a pseudo-class.

THE CONTROL MUST COME FIRST

`~` is the general sibling combinator. **CSS cannot look backwards**, so a `peer` after the thing it styles does nothing at all.

CYCLE 3 · BEFORE YOU TYPE

“ISN'T THIS INLINE STYLES?” NO: A CLASS CAN CARRY A CONDITION

```
1 @media (hover: hover) {  
2   .hover\:bg-brand-700:hover { background-color: ... } }  
3  
4 @media (width >= 48rem) {  
5   .md\:px-8 { padding-inline: ... } }  
6  
7 @media (prefers-color-scheme: dark) {  
8   .dark\:bg-brand-300 { background-color: ... } }
```

An attribute is a value. These are conditions — and each prefix (`hover:`, `md:`, `dark:`) is a variant.

ASK BOTH OF THEM TO DO FOUR THINGS

a style attribute and a utility, side by side

reload

open

narrow the frame

dark

add an overriding class

viewport is >= md (48rem)

style=""

utilities

Deploy

Deploy

asked to...	style=""	utilities	measured right now
change on hover	no	yes	a style attribute has no :hover to hang a value on
change at a breakpoint	no	yes	a style attribute has no @media
change in dark mode	no	yes	same reason – no @media
be overridden by a later class	no	yes	the attribute is (1,0,0,0) – it beats every selector you can write
hold a value computed at run time	yes	no	THE one thing the attribute is genuinely for – utilities are a fixed set

right now:

style="" button background rgb(24, 45, 242) · padding 8px 16px

utility button background rgb(24, 45, 242) · padding 8px 32px

and here is what those utility classes actually ARE. Read out of document.styleSheets – this is the browser answering, not a lookup table:

▶ **HANDS-ON** `demos/02-not-inline-styles/` — press the buttons in order: **HOVER · NARROW · DARK · OVERRIDE**. In each step, watch the left button.

WHAT A STYLE ATTRIBUTE IS GENUINELY FOR

A VALUE COMPUTED AT RUN TIME

A progress bar at 63%. A tooltip at the cursor. A number that did not exist when the page was written. Utilities are a fixed set generated from the source.

THAT IS THE WHOLE LIST

And it is worth saying, because a rebuttal with a boundary is stronger than one without. You will use it in week 10.

CYCLE 3 · BEFORE YOU TYPE

A VARIANT GOES IN FRONT OF A UTILITY: variant:utility

For example `hover:bg-brand-100`, `focus-visible:outline-2`,
`disabled:opacity-50`, `motion-safe:transition-colors`.

VARIANT	GENERATES	NOTE
<code>hover:</code>	<code>@media (hover: hover) + :hover</code>	never sticks on a phone — free
<code>focus-visible:</code>	<code>:focus-visible</code>	the rubric line. Keyboard only
<code>focus:</code>	<code>:focus</code>	fires on mouse too. Usually you want the one above
<code>active:</code>	<code>:active</code>	only while the button is held down
<code>disabled:</code>	<code>:disabled</code>	a state, not a fifth variant
<code>dark:</code>	<code>@media (prefers-color-scheme: dark)</code>	the OS preference. No button
<code>motion-safe:</code>	<code>@media (prefers-reduced-motion: no-preference)</code>	put every transition behind it
<code>placeholder:</code>	<code>::placeholder</code>	pseudo-elements work too



CYCLE 3 · BEFORE YOU TYPE

A DARK RAMP IS A SECOND SET OF DECISIONS, NOT AN INVERSION

WHAT PEOPLE EXPECT

Flip the background, flip the text, ship it.

WHAT ACTUALLY HAPPENS

`brand-500` on near-black is **2.36:1** — measured in cycle 3. Under 3 for a ring, under 4.5 for a word.

WHAT YOU DO

A lighter cut of the same colour.
4.5:1 in both ramps, independently.

And no toggle: `dark:` is a media query on a preference the user already set.

BOTH RAMPS, ON SCREEN AT ONCE

the same card, two ramps, contrast computed live

LIGHT RAMP

GATEWAY

Certificate expires in 6 days

Renew it before Friday or the gateway starts refusing connections.

Renew now

DARK RAMP

GATEWAY

Certificate expires in 6 days

Renew it before Friday or the gateway starts refusing connections.

Renew now

every colour in both cards, resolved from the rendered page – and what week 3 called it

@theme token	week 3 (tokens.css)	light	dark
--color-page	--color-bg	rgb(255, 255, 255)	rgb(16, 20, 30)
--color-surface	--color-surface	rgb(248, 249, 252)	rgb(25, 30, 42)
--color-edge	--color-border	rgb(199, 205, 219)	rgb(63, 71, 90)
--color-ink	--color-text	rgb(15, 23, 41)	rgb(242, 245, 248)

▶ **HANDS-ON** `demos/07-dark/` — ask yourself “**WHAT CHANGED?**” Give it fifteen seconds.

CYCLE 3 · 30 MIN

ONE LINK, FOUR STATES, AND A SECOND RAMP

A link and a button get every state the assignment checks: a hover, a ring that appears only from the keyboard, a transition that does not exist for someone who asked for less motion, a disabled look. Then dark mode — and the brand colour that was fine in light is 2.36:1 in dark.

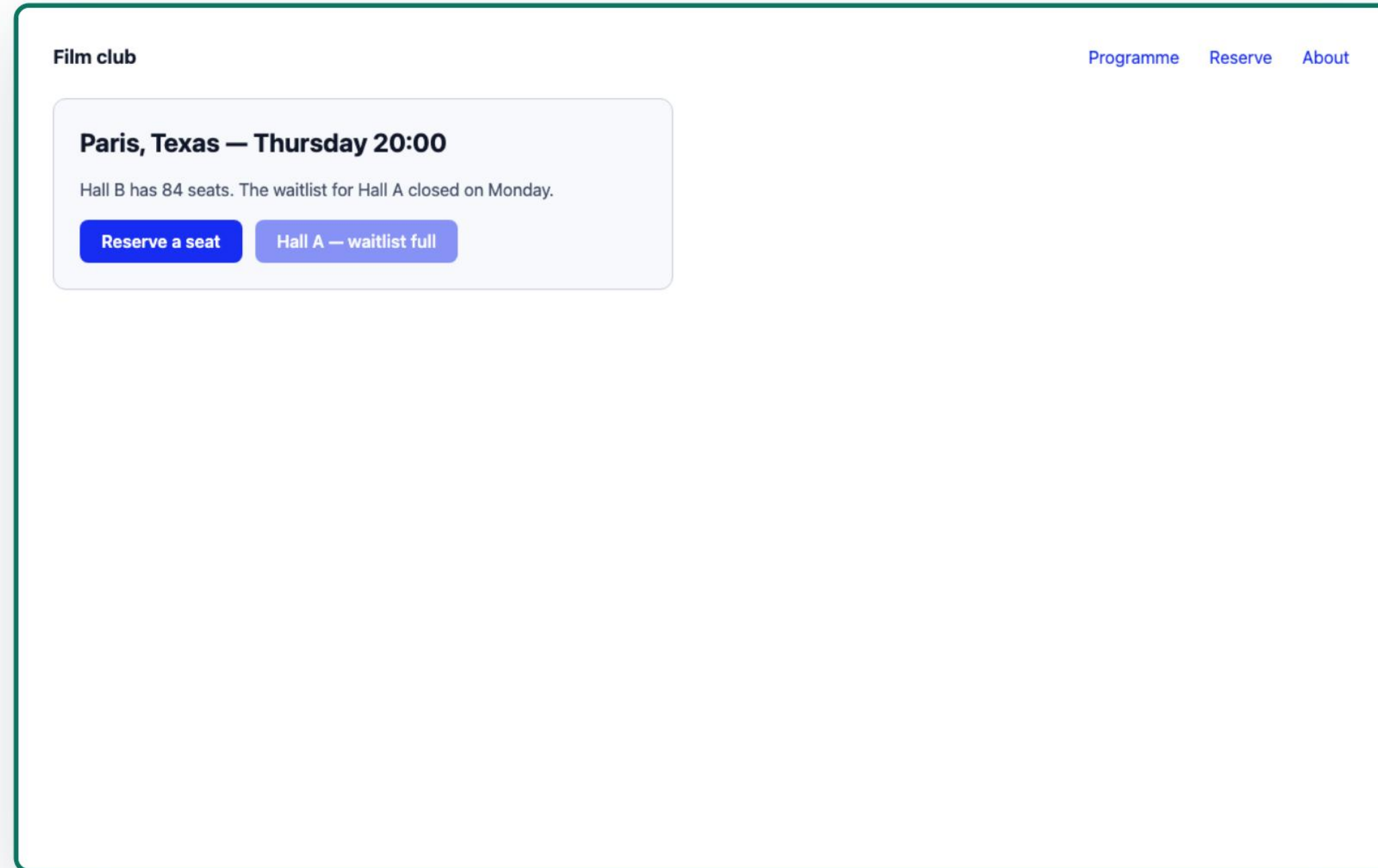
1 I DO13 MIN 6 on the slides above, 7 in the editor · live-code/03-link-four-states · hover: · focus-visible:outline-2 · motion-safe:transition-colors · disabled: · dark: as an inversion, then as a second ramp	2 WE DO7 MIN you type it on your copy, brand-300 in dark from the start · Tab with the mouse down, Rendering in dark and with reduce · compare with checkpoint 3	3 YOU DO8 MIN a fourth link and a card hover, in both schemes · the colour picker, Tab, Rendering	4 DEBRIEF2 MIN focus: against focus-visible: · 2.36 against 7.22 · deeper slides
---	--	---	--

YOU DO · AI-FREE



CHECKPOINT 3

YOUR SCREEN SHOULD LOOK LIKE THIS



Compare — `live-code/03-link-four-states/checkpoint.png` is this picture at 1280; `live-code/03-link-four-states/end/` is the code behind it. Tab with the mouse down: a solid 2px ring on every link. Rendering in dark: the links in light indigo, 7.22:1. With `prefers-reduced-motion: reduce`: `transition-duration: 0s`.

BREAK

10 MINUTES

Back at 100. Cycle 4 is the AI cycle — have a chat window open, free or paid, Live Server running, and `ai-lab/04-three-answers-one-prompt/artifact/start/` in a tab.

CYCLE 4 · BEFORE YOU START

WHAT YOU WILL SEE IN TUTORIALS, AND WHAT A CDN IS

TAILWIND V3 — FROM A CDN, THE WAY MOST TUTORIALS SHOW IT

```
<script src="https://cdn.tailwindcss.com"></script>
<script>
  tailwind.config = {
    theme: {
      extend: {
        colors: { brand: '#182df2' },
      },
    },
  };
</script>
```

V4 — WHAT WE USE

```
<script src="vendor/tailwind.js"></script>
<style type="text/tailwindcss">
  @theme {
    --color-brand-500: hsl(234 89% 52%);
  }
</style>
```

A **CDN** is a public server that hands your page a file from a URL. Ours sits in `vendor/`, so it works with the network off, as in week 13. The class names barely moved; only the config did.

Measured on our v4 file: the v3 config line throws `tailwind is not defined` and white text sits on nothing — 1.00:1.

CYCLE 4 · 25 MIN

THREE ANSWERS TO ONE PROMPT

One naive request — "Tailwind from a CDN, responsive, in my brand colour" — sent in three chats. Three answers, all well explained. Run them the way your project runs: one of them keeps its colour in a file this week's Tailwind never reads. Choosing between the other two is what you write down.

1 I DO

8 MIN

3 on the slides above, 5 in the chat · prompt P1 in three chats, live · run each in artifact/start/, vendor/tailwind.js, no CDN

2 WE DO

6 MIN

everyone sends P1 · run what came back at 375, 1280, with Tab and the console · find the one that breaks, and under which condition, before it is named

3 YOU DO

9 MIN

write your own prompt, P2, as checkable constraints · run it offline · the review exemplar on screen · then the three parts: chosen, broken, the correct one rejected

4 DEBRIEF

2 MIN

right for another setup is wrong for yours · the review exemplar

CYCLE 4 · BEFORE YOU WRITE

WHAT A REVIEW LOOKS LIKE, BEFORE YOU WRITE ONE

AI-REVIEW-GOOD.MD

Claim and conclusion apart. "The explanation says two columns below 1024. *I checked at 700:* three columns, 217px each." Every sentence names the width, the number, the tool. The longest part is why the **correct** version was rejected — and when it would have been chosen.

AI-REVIEW-BAD.MD

Same three versions. "It looked cleanest." "I saw it in the code." "Grid is not allowed this week." True, short, and worth nothing: no width, no measurement, no cost, no "when would I pick it".

Three parts: what you chose and why · what is broken, how you found it, under which condition · the correct one you rejected, with something in the code and what it costs.

CHECKPOINT 4

THREE QUESTIONS BEFORE THE NEXT CYCLE

1. Did you **run** every answer from `vendor/tailwind.js`, with the CDN line left out — and not stop because it looked right with the network on?
2. Is every constraint in your P2 something the browser can check — where Tailwind loads from, where the colour lives, `md:` and no `max-*`, `focus-visible:`, `motion-safe:` — and does what came back meet all of them offline?
3. Does your part on "the correct one I rejected" name **something in the code and what it costs** — "the brand colour written seven times", not "less clean"?

Compare — `ai-lab/04-three-answers-one-prompt/checkpoint-he.md` has the same three questions and the measurements; `artifact/answer-a/` to `answer-p2/` are the four answers, ready to open side by side.

CYCLE 5 · BEFORE YOU TYPE

“CLASS SOUP IS JUST WHAT THIS LOOKS LIKE.”

six copies of one button, and what happened to them

reloadopen

TODAY SIX COPIES

The same button, typed out six times. Find the odd ones out by eye before you read the audit.

RenewDeployScaleRestart

Roll backArchive

```
<button class="cursor-pointer rounded-lg bg-brand-500 px-5 py-2 text-sm font-semibold text-white hover:bg-brand-700">Renew</button>
```

...and five more, each typed out again

6 buttons, meant to be identical.
distinct renderings: 2
carrying a hover rule: 5 of 6
renders differently: Scale
has no hover state: Roll back

Two of the six have already drifted, and neither drift was a decision.
One has px-4 where the rest have px-5 – four pixels, invisible in a row.

THIS WEEK ONE SOURCE

The button is defined once, on components.html, and every use is a copy of that — by hand, no build step. That is what CA6 asks for. On this page the six are generated from one string, so that the demo has a single source too; the discipline is the same and the audit cannot tell them apart, which is the point.

RenewDeployScaleRestart

Roll backArchive

components.html – the ONE definition:

```
<!-- button / primary -->  
<button class="cursor-pointer rounded-lg bg-brand-500 px-5 py-2 text-sm font-semibold text-white hover:bg-brand-700">Label</button>
```

everywhere else: copy that line.
changed your mind? change it there, then re-copy.

6 buttons, meant to be identical.
distinct renderings: 1
carrying a hover rule: 6 of 6

WEEK 10 ONE FUNCTION

Read this one; do not write it yet. It needs template literals and functions, which is week 8, and the render pattern it belongs to is week 10. It is here so you know the ladder has a top.

RenewDeployScaleRestart

Roll backArchive

const BUTTON =
'cursor-pointer rounded-lg bg-brand-500 ' +
'px-5 py-2 text-sm font-semibold text-white ' +
'hover:bg-brand-700';

const button = (label) =>
`<button type="button" class="\${BUTTON}">\${label}</button>`;

LABELS.map(button).join('')

6 buttons, meant to be identical.
distinct renderings: 1
carrying a hover rule: 6 of 6

One rendering, and now it is not a discipline – it is

► **CLASS SOUP** is the name for long class lists copied again and again. **HANDS-ON** demos/09-extraction/ — panel one. Ask the room to find the odd button **BY EYE** before reading the audit.

50

CYCLE 5 · BEFORE YOU TYPE

“@apply SOLVES THIS.”

the same four measurements, on an @apply stylesheet↻ reload↗ open

THE SAME BUTTONS, WRITTEN WITH @APPLY

Renew

Delete

Archive

and the same two buttons again, inside .panel — which is a real wrapper in a real page and not a trick:

Renew

Delete

Archive

```
.btn      { @apply cursor-pointer rounded-lg bg-brand-500 px-5 py-2 ... }
.btn-danger { @apply bg-red-600; }

.panel .btn { @apply bg-brand-500; } /* written later, by someone else */
```

the Delete button, measured in both rows:

```
outside .panel  oklch(0.577 0.245 27.325)
inside  .panel  oklch(0.577 0.245 27.325)
```

they agree — only because .panel .btn.btn-danger was written to win the fight that .panel .btn started. Delete that one line and the Delete button turns brand-blue inside panels and nowhere else.

Utilities cannot do this to you. `bg-red-600` is one class at (0,1,0) and it

► **HANDS-ON** `demos/10-apply-trap/` — `@apply` copies a list of utilities into a rule of your own. Write `.btn { @apply ... }` on the board first. Ask whether it beats six copies.

CYCLE 5 · 20 MIN

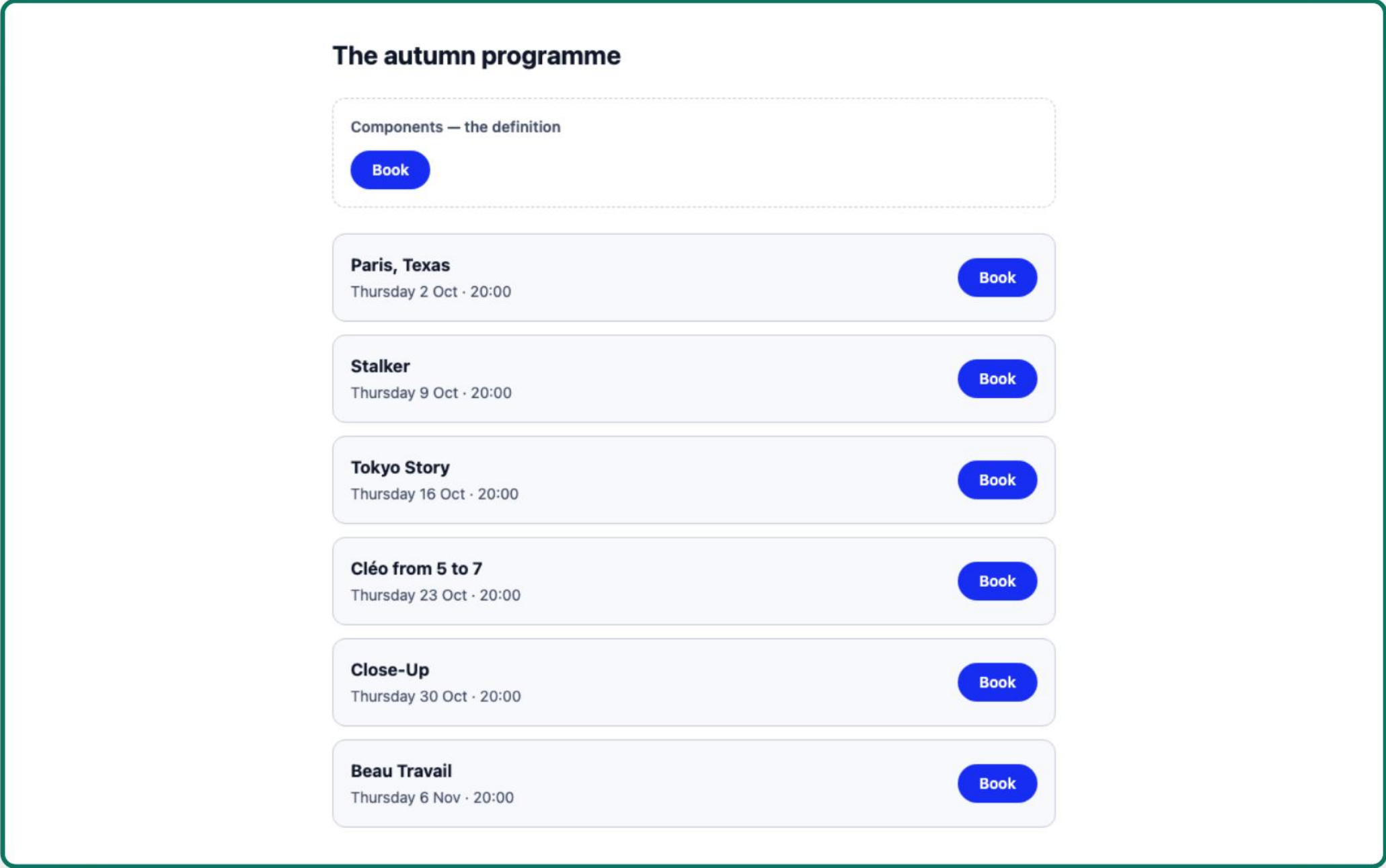
ONE BUTTON, SIX COPIES

A button that appears in six places is defined once, in a gallery at the top of the page, and every copy is **pasted** from there, because copies typed from memory drift. The ladder: copies, one definition you copy from (today), one function (week 10).

1 I DO7 MIN 2 on the slides above, 5 in the editor · live-code/05-one-button-six-copies · find the two drifted copies, by eye, then one console line (copy this) · paste from the gallery · a sixth typed from memory · paste again · rounded-full, seven times	2 WE DO5 MIN copy this: you paste, never type, the sixth one too · the console line, then hover and Tab on every button · compare with checkpoint 5	3 YOU DO6 MIN the screening card as a component: defined in the gallery, pasted six times · AI as assistant, checked in the console	4 DEBRIEF2 MIN replace, do not repair · what @apply solves and what it leaves · deeper slides
---	---	---	---

CHECKPOINT 5

YOUR SCREEN SHOULD LOOK LIKE THIS



Compare — `live-code/05-one-button-six-copies/checkpoint.png` is this picture at 1280; `live-code/05-one-button-six-copies/end/` is the code behind it. Six buttons, six hovers, six `solid 2px` rings on Tab — and every class list on the page pasted from the gallery (**copy this**, never typed).

AND THE ONE PLACE IT IS RIGHT

WHEN YOU DO NOT OWN THE MARKUP

A base rule for every `<a>` in prose. A `::selection` colour. A third-party widget you cannot put a class on.

A HANDFUL OF RULES, IN A BASE LAYER

That is the whole legitimate use. If you want it everywhere, what you want is a **component**.

Which is why `@apply` is forbidden in the assignment: it would let you avoid the thing the assignment is about.

REPEATED MARKUP – THE THREE RUNGS

RUNG	WHAT IT IS	WHAT IT COSTS
copies	the same class list typed out N times	they drift within a fortnight, silently
a gallery page	defined once, copied from there	a discipline, so it can be broken. This is the assignment, and cycle 5
a function	one string, N calls	nothing. Week 10 , and it needs JavaScript
<code>@apply</code>	not a rung. Moves styles into a stylesheet	every cost of having a stylesheet, back

`@apply` 's one legitimate use: elements whose markup you do not control — prose links, `::selection`, a third-party widget.

THE FIVE, AND HOW TO CATCH EACH IN YOUR OWN FILE

1. "IT'S INLINE STYLES"

DevTools → Rendering → emulate

`prefers-color-scheme: dark` ·

A `dark:` · class answers; a

`style=""` cannot.

2. MEMORISING THE CLASS NAMES

Hover the class in VS Code: the Tailwind extension the repo recommends shows the CSS it generates.

3. `@apply` EVERYWHERE

Search your CSS for `@apply`. More than the one legitimate base-layer use hands back the stylesheet you were leaving.

4. IGNORING THE SCALE

Search your HTML for `-[` ·. Every arbitrary value needs a reason you can say; `p-[13px]` · beside `p-3` has none.

5. CLASS SOUP WITH NO COMPONENT

Search for your button's full class string. More than one exact hit is a copy waiting to drift.

ASSIGNMENT · BEFORE TINDOG

A PICTURE KEEPS ITS SHAPE: FIX ONE SIDE, LET THE OTHER FOLLOW

A PHOTO: `w-full h-auto`

The width fills the column and `h-auto` lets the height follow, so the ratio is kept. Never fix both sides.

LOGOS:

`h-8 w-auto object-contain`

Fix the height, let the width follow: four logos of different sizes agree on one height and none is squashed. `object-contain` fits the picture inside its box without cropping.

A FIXED BOX:

`aspect-square object-cover`

The box has a shape, and `object-cover` fills it and crops the overflow. Week 5's `aspect-ratio` and `object-fit`, as classes.

TinDog's checks are these: the hero photo keeps its ratio at 375 and 1280, and the logos line up without distortion.

ASSIGNMENT · 145–175

THE ASSIGNMENT: ASSEMBLE WHAT YOU TYPED, THEN EXTEND IT

ASSEMBLE · CORE

Last week's dashboard with no stylesheet: the cards in utilities on the scale (cycle 1) · the shell in twelve columns, the main column before the menu on a phone (cycle 2) · hover and a `focus-visible` ring on the side menu, and a dark ramp that passes 4.5:1 in both schemes (cycle 3) · and the written review of the three AI versions in `variants/` (cycle 4). Every `CODE-HERE` marker in the starter is a class list you write.

EXTEND · STRETCH AND CHALLENGE

`components.html`: button, card, badge and field defined once and pasted everywhere (cycle 5) · the broken AI variant fixed in its file · then TinDog, rebuilt with nothing typed from memory.

THE RULES, AND THE EXAM

No stylesheet of your own, no `style=""`, no `@apply`, no `!important`. Tailwind from `vendor/tailwind.js`, never a CDN. AI as assistant: you choose, and say in writing what you rejected. Repo URL and SHA in Moodle. The exam in week 13 asks about this code, offline.

DEBRIEF · 175–180

THREE QUESTIONS BEFORE YOU GO

1. `order-first` on the main column. First of what — and where does Tab go first?
2. Two answers from Claude both work. What do you write about the one you did not take?
3. The same button in six places. Why is `@apply` not the fix, and what is?

► **GO DEEPER** — every slide marked *Deeper · self-study* is optional depth and is not examined: the full translation tables, the demos behind the cycles, the reset Tailwind brings, the selectors behind `group-` and `peer-`, the three rungs of repeated markup, and where to look things up. Each one sits after the slide it extends, or after its cycle.

WHERE TO LOOK, IN ORDER

- 1 **demos/03-translator/**
— paste CSS, get the class. It measures.
- 2 **Tailwind CSS IntelliSense**
— and it flags a class that does not exist.
- 3 **tailwindcss.com/docs**
— the search takes CSS property names.
- 4 **DevTools**
— the styles panel. It is ordinary CSS and always was.

Starting at the docs is memorising.
Starting at the translator is translating.